A Discussion

At a high level, our claims characterize neural architectures endowed with CoT reasoning in terms of well-understood *probabilistic* models of computation, giving the framework of CoT reasoning a novel probabilistic perspective. This allows the reconciliation of theoretical results about neural LMs with the general probabilistic language modeling framework. In doing so, we put existing results about the representational capacity of LMs into perspective and show how they can be thought of as performing "CoT reasoning in disguise." Perhaps surprisingly, the inclusion of CoT reasoning steps results in a natural inclusion of non-determinism in otherwise deterministic neural LMs, as exemplified by Thm. 4.1. Outputting the states as part of the generated string—whose generation is inherently non-deterministic—allows a neural LM to keep track of the trajectory of the current execution. This suggests that CoT reasoning might provide an interesting avenue for exploring the non-deterministic representational capacity of neural LMs. Note that this means CoT reasoning can endow neural LMs with higher expressivity since, e.g., non-deterministic PFSAs are strictly more expressive than deterministic ones. Similarly, our result that RNNs or transformer LMs with CoT reasoning can simulate PTMs rather than regular Turing machines is important as it allows *probabilistic* computation using such neural LMs. This means one can sample from them multiple times to assess the certainty of string inclusion in a language. Furthermore, the problems LMs could solve efficiently can be described by different complexity classes such as BPP, ZPP, etc. instead of P. [18]

B Related Work

Turing completeness of RNNs. Plenty of existing work has investigated the representational capacity of RNNs, both as recognizers as well as LMs (e.g., McCulloch and Pitts, 1943; Kleene, 1956; Siegelmann and Sontag, 1992; Hao et al., 2018; Korsky and Berwick, 2019; Merrill, 2019; Merrill et al., 2020; Hewitt et al., 2020; Chung and Siegelmann, 2021; Merrill et al., 2022; Merrill and Tsilivis, 2022; Svete and Cotterell, 2023; Nowak et al., 2023, *inter alia*). Most relevant to our work, Siegelmann and Sontag (1992); Chung and Siegelmann (2021) show how RNNs with unbounded precision and unbounded computation time can simulate Turing machines and discuss the implications. Nowak et al. (2023) extend this to the probabilistic setting, showing that RNN LMs can emulate certain PTMs, but require the LMs to be able to perform non-emitting steps.

Turing completeness of transformers. Pérez et al. (2021) show the Turing completeness of hard attention encoder–decoder transformer by encoding the configuration of the Turing machine in the output of the transformer. Bhattamishra et al. (2020) provide a different perspective on Turing completeness of the architecture by showing that transformers can simulate RNNs. Turing completeness is in this sense a simple consequence of the Turing completeness of RNNs. Bhattamishra et al.'s (2020) construction, however, relies on emitting symbols from an (uncountably) infinite set (rational-valued vectors), in contradiction to the requirement of the LM working over a *finite* alphabet. In this sense, they make use of the so-called *regression* transformer setup, which does not lend itself well to the discrete language modeling (Von Oswald et al., 2023). In concurrent work, Merrill and Sabharwal (2024) adapt Pérez et al.'s (2021) result to the CoT setting, connecting Turing completeness of transformer decoders to CoT reasoning similar to our work. In contrast to our work, they make statements about the model's ability to decide language membership (simulating a Turing machine that accepts or rejects an input) and not to represent probabilistic languages. In that sense, the transformer they construct is not a language model. Besides being probabilistic, our construction also enables the analysis of any autoregressive LM architecture, and we focus on RNN and transformer LMs. Feng et al. (2023) similarly show that CoT transformers can perform arithmetic expressions and dynamic programming by generating intermediate results, backing these claims up with experimental evidence, but stopping short of showing Turing completeness. Du et al. (2023) provide a related result, showing that transformer LMs with continuous transformation functions are *tight*. This might at first glance contradict the results on Turing completeness, since the latter might require the model to run indefinitely, resulting in a non-tight model. However,

[18] For a comparison of these complexity classes, see e.g. Papadimitriou (1994).

the discrepancy is resolved by noting that the tightness result crucially relies on the use of the softmax normalization function over the reals $\mathbb{R}$ (without $\pm\infty$) in Eq. (2). This, together with the encoding function of the transformer, results in a non-diminishing probability of generating EOS and thus in a tight model. Our use of the sparsemax function sidesteps this issue by allowing us to assign EOS probability 0.

C Additional Preliminaries

In our construction, we assume that strings are prefixed with the beginning-of-string symbol BOS. This is to give a base case for recurrent definitions such as Def. 2.1. This is just for ease of notation in our proofs. Note that instead of $p(\overline{y} \mid \text{BOS})$, we could also equivalently write $p(\overline{y} \mid \varepsilon)$.

C.1 Turing Machines

Under our definition, a probabilistic Turing machine has two tapes.[19] The first is the **processing tape** on which symbols from the tape alphabet can be read and written. The second is a write-only **output tape** for symbols of the output alphabet. In the beginning, the processing tape contains the designated $\perp$ symbol in the leftmost cell while all other cells contain the blank symbol $\sqcup$. The output tape is empty at the beginning. Starting in the initial state q_ι, at each time step t, a transition is sampled out of all available transitions for the given state q and the current working tape symbol γ, and then applied. The sampling happens according to the transition probability w (recall that the transition weights w are non-negative and sum to 1 for each pair of state q tape symbol γ). We write transitions as $(q, \gamma) \xrightarrow{y,d/w} (q', \gamma')$, where $q, q' \in Q, \gamma, \gamma' \in \Gamma, y \in \Sigma_\varepsilon$, $w \in \mathbb{Q}_{\geq 0}$, and $d \in \{L, R\}$, with the following interpretation. When the machine is in state q and its head is reading γ on the working tape, it moves to state q', writes γ' to the working tape, writes y to the output tape if $y \in \Sigma$ or nothing if $y = \varepsilon$, and it moves the head on the working tape by one symbol in the direction d, i.e., to the left (L), or to the right (R). As with any non-deterministic machine, the above definition naturally gives rise to the notion of a tree of possible computations (Sipser, 2013, p. 48). A **branch** $\boldsymbol{\pi}$ of the computation tree of a PTM is a sequence of consecutive transitions

$$(q_\iota, \perp) \xrightarrow{y_1,d_1/w_1} (q_2, \gamma'_1), (q_2, \gamma_2) \xrightarrow{y_2,d_2/w_2} (q_3, \gamma'_2), \ldots, (q_N, \gamma_N) \xrightarrow{y_N,d_N/w_N} (q_{N+1}, \gamma'_N) \tag{12}$$

We say that a branch is **accepting** if the branch reaches the final state q_φ.[20] The **yield** of an accepting computation branch[21] is the sequence of symbols $\boldsymbol{y} \in \Sigma^*$ written on the output tape at that point in the computation, i.e., a concatenation of the (non-ε) symbols $y_1, \ldots, y_N$, where $N = |\boldsymbol{\pi}|$ is the number of transitions in the branch. The **weight** of an accepting branch is the product of the weights of its transitions, i.e.,

$$\mathrm{w}(\boldsymbol{\pi}) \stackrel{\text{def}}{=} \prod_{n=0}^{N} w_n. \tag{13}$$

We denote the branches that yield a given string $\boldsymbol{y}$ by $\Pi(\mathcal{M}, \boldsymbol{y})$. The sum of weights of all branches that yield a certain string $\boldsymbol{y} \in \Sigma^*$ is the **stringsum** of that string, defined as

$$\mathcal{M}(\boldsymbol{y}) \stackrel{\text{def}}{=} \sum_{\boldsymbol{\pi} \in \Pi(\mathcal{M}, \boldsymbol{y})} \mathrm{w}(\boldsymbol{\pi}). \tag{14}$$

This definition gives rise to a semimeasure over strings whose sum over all possible strings is exactly the halting probability[22] of the PTM, i.e., the probability that starting from the initial state q_ι, $\mathcal{M}$ reaches a final state q_φ

$$\sum_{\boldsymbol{y} \in \Sigma^*} \mathcal{M}(\boldsymbol{y}) = \mathbb{P}(\mathcal{M} \text{ halts}). \tag{15}$$

[19] Adding another tape does not increase the computational power of a Turing machine (Sipser, 2013, Ch. 3).

[20] The final state has no outgoing transitions.

[21] We only consider branches that end in a final state when discussing the yield and weight of branches. This means we only take into account finite instances of computation.

[22] The halting probability is the probability that the execution of the PTM will end in a halting state after finitely many steps.

C.2 Pushdown Automata

Definition C.1. *A **probabilistic pushdown automaton** (PPDA) is a tuple* $(Q, \Sigma, \Gamma, \delta, (q_\iota, \mathrm{S}), (q_\varphi, \varepsilon))$ *where* Q *is a finite set of states,* Σ *is the input alphabet,* Γ *is the stack alphabet,* $\delta \subseteq Q \times \Gamma \times \Sigma_\varepsilon \times \mathbb{Q}_{\geq 0} \times Q \times \Gamma^*$ *is a finite set of weighted transitions, and* (q_ι, S) *and* (q_φ, ε) *are called the initial and final configuration, respectively. Moreover, for all states* $q \in Q$ *and stack symbols* $\gamma \in \Gamma$*,* δ *satisfies* $\sum_{(q,\gamma) \xrightarrow{y/w} (q',\boldsymbol{\gamma}) \in \delta} w = 1.$

Our definition of PPDA is similar to that of Abney et al. (1999), but, unlike theirs, our machine must end its computation in the final configuration rather than just empty the stack. We write transitions $(q, \gamma, y, w, q', \boldsymbol{\gamma}) \in \delta$ as $(q, \gamma) \xrightarrow{y/w} (q', \boldsymbol{\gamma})$ which represent a move with weight w from state q to q' while scanning or outputting y, popping the symbol γ from the stack and pushing the sequence of symbols $\boldsymbol{\gamma}$.

Definition C.2. *A **configuration*** $(q, \boldsymbol{\gamma}) \in Q \times \Gamma^*$ *is a pair containing the current state and the current contents of the stack.*

A PPDA $\mathcal{P} = (Q, \Sigma, \Gamma, \delta, (q_\iota, \mathrm{S}), (q_\varphi, \varepsilon))$ is called **deterministic** if for every $q \in Q$, $\gamma \in \Gamma$ and $y \in \Sigma_\varepsilon$, there is at most one transition $(q, \gamma) \xrightarrow{y/w} q', \boldsymbol{\gamma}$ with $w > 0$. Additionally, if there is a transition $q, \gamma \xrightarrow{y/w} q', \boldsymbol{\gamma}$ such that $y \in \Sigma$, then there is no transition $q, \gamma \xrightarrow{\varepsilon/w'} q'', \boldsymbol{\gamma}'$ with $w' > 0$. Intuitively, there exists at most one next move given any configuration in a deterministic PPDA. A **run** $\boldsymbol{\pi}$ of a PPDA is a sequence of configurations and transitions,

$$(q_0, \boldsymbol{\gamma}_0), \tau_1, (q_1, \boldsymbol{\gamma}_1), \ldots, \tau_N, (q_N, \boldsymbol{\gamma}_N), \tag{16}$$

where $(q_n, \boldsymbol{\gamma}_n)$ is the confguration reached by taking transition τ_n from configuration $\left(q_{n-1}, \boldsymbol{\gamma}_{n-1}\right)$ for any $n \in [1, N]$. If $(q_0, \boldsymbol{\gamma}_0) = (q_\iota, \mathrm{S})$ and $(q_N, \boldsymbol{\gamma}_N) = (q_\varphi, \varepsilon)$, we call $\boldsymbol{\pi}$ **accepting**. The **yield** of an accepting run is $\mathbf{s}(\boldsymbol{\pi}) \stackrel{\text{def}}{=} y_1 \cdots y_N$, where y_n is the symbol scanned by τ_n. The **weight** of an accepting run, $\mathrm{w}(\boldsymbol{\pi})$, is the product of the weight of its transitions,

$$\mathrm{w}(\boldsymbol{\pi}) \stackrel{\text{def}}{=} \prod_{n=1}^{N} w_n, \tag{17}$$

where w_n is the weight of transition τ_n. We write $\Pi(\mathcal{P}, \boldsymbol{y})$ for the set of all accepting runs with yield $\boldsymbol{y}$. The sum of the weights of all accepting runs of some PPDA $\mathcal{P}$ that yield a certain string $\boldsymbol{y} \in \Sigma^*$ is called the **stringsum** and is defined as

$$\mathcal{P}(\boldsymbol{y}) \stackrel{\text{def}}{=} \sum_{\boldsymbol{\pi} \in \Pi(\mathcal{P}, \boldsymbol{y})} \mathrm{w}(\boldsymbol{\pi}). \tag{18}$$

C.3 Two-stack Pushdown Automata

Definition C.3. *A **two-stack pushdown automaton** (2PDA) is a machine specified by the* 6*-tuple* $\mathcal{P} = (Q, \Sigma, \Gamma, \delta, q_\iota, q_\varphi)$*, where* Q *is a finite set of states,* Σ *is an alphabet of input symbols,* Γ *is an alphabet of stack symbols, including the bottom-of-stack symbol* $\perp$*,* $\delta \colon Q \times \Gamma \times \Sigma_\varepsilon \times \mathbb{Q}_{\geq 0} \times Q \times \Gamma_\varepsilon^4$ *is a finite set of rationally-weighted transitions, and* q_ι *and* q_φ *are the initial and the final state, respectively.*

We use a definition similar to that of Nowak et al.'s (2023), which assumes without loss of generality that transitions are determined by the current state and the top symbol of the first stack only. See Nowak et al. (2023, Appendix B) for a proof. We write transitions as $q \xrightarrow[\gamma_2 \to \gamma_4]{\gamma, y, \gamma_1 \to \gamma_3 / w} q'$, which represent a move with weight w from state q, with the top of the first stack γ, to state q', while scanning or outputting y, popping γ_1 and γ_2 from the first and second stack, respectively, and pushing γ_3 and γ_4 to the first and second stack, respectively. A 2PDA is called **probabilistic** if, for every state q and top symbol γ of the first stack, the weights of the transitions define a probability distribution, i.e.,

$$\sum_{q \xrightarrow[\gamma_2 \to \gamma_4]{\gamma, y, \gamma_1 \to \gamma_3 / w} q' \in \delta} w = 1. \tag{19}$$